

MEDIUM

Vertically Centered Login Card

A login page should show a card centered both horizontally and vertically in the browser window. The developer applied ``flex`` to the full-height wrapper but the card is still stuck at the top-left corner.

```
<div class="flex h-screen bg-gray-100">
  <div class="w-96 bg-white p-8 rounded-lg shadow-md">
    <h2 class="text-xl font-bold">Sign in</h2>
    <p>Email and password fields go here.</p>
  </div>
</div>
```

Your task:

1. Explain why the card is not centered given the current classes.
2. Add the minimum utility classes needed to center the card both horizontally and vertically.
3. Keep `h-screen` on the wrapper.

MEDIUM

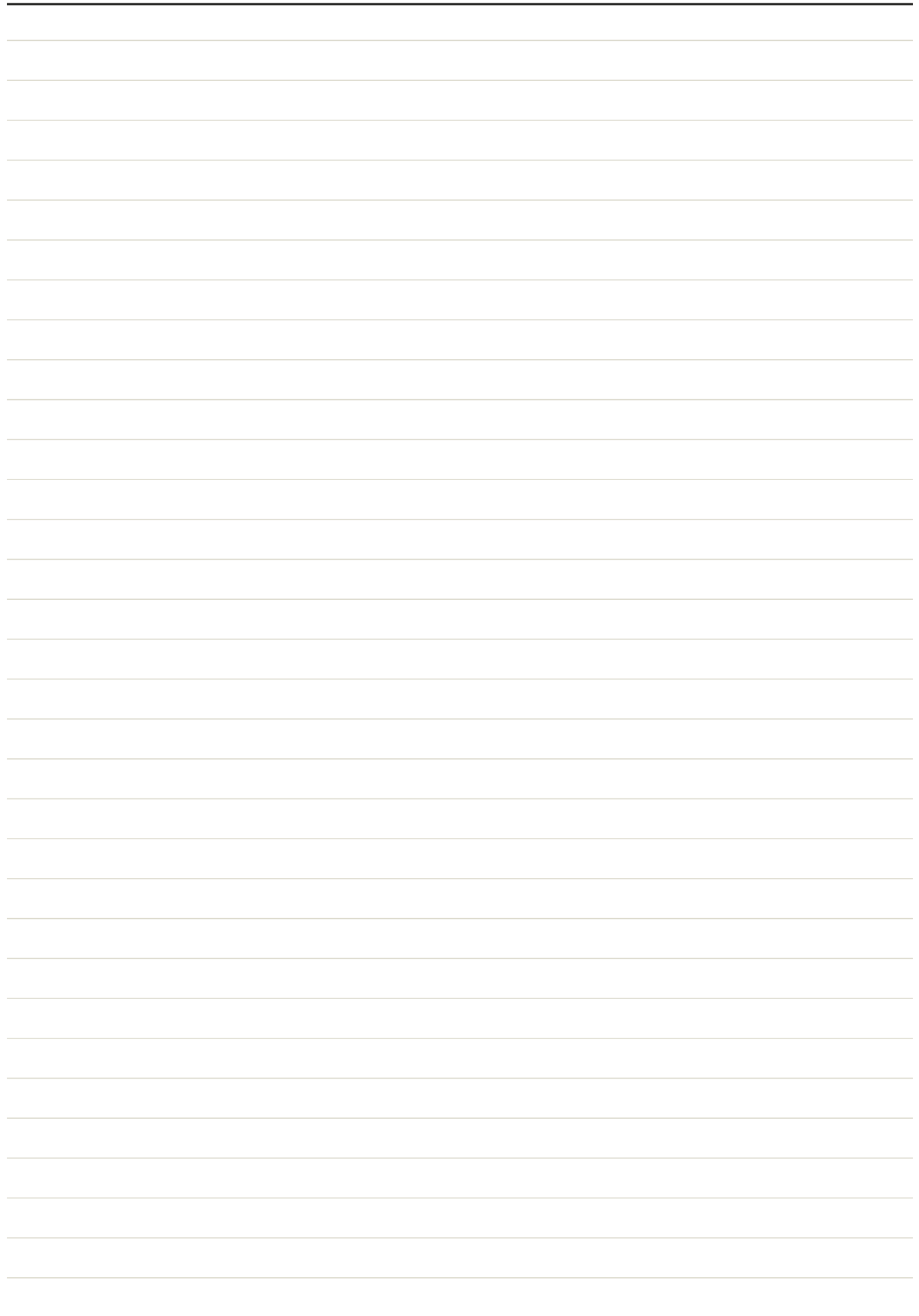
Responsive Photo Gallery Grid

A photo gallery currently always shows 4 columns via ``grid-cols-4``, which squashes thumbnails into unreadably thin strips on mobile phones. It should show 1 column on small phones, 2 columns on tablets, and 4 columns on desktop.

```
<div class="grid grid-cols-4 gap-4">
  
  
  
  
</div>
```

Your task:

1. Rewrite the grid classes so the layout is mobile-first: 1 column by default.
2. Add a breakpoint that switches to 2 columns for tablet-sized screens.
3. Add a breakpoint that switches to 4 columns for large desktop screens.
4. Keep a consistent gap between images at every size.



MEDIUM

Card Component with Variant Modifiers

A design system needs a reusable card pattern. All cards share the same base look (white background, rounded corners, shadow, padding), but a 'featured' card needs a highlighted border and stronger shadow, and a 'compact' card needs reduced padding and smaller text for dense lists.

```
<div class="p-6 rounded-lg shadow bg-white">
  <h3 class="font-bold">Plan Name</h3>
  <p>Plan description text.</p>
</div>
```

Your task:

1. Write markup for a base card using the given classes.
2. Write a 'featured' variant that adds a colored border and a stronger shadow while keeping the base look.
3. Write a 'compact' variant that reduces padding and text size compared to the base.

MEDIUM

Button Variant System

A team needs three buttons — primary, secondary, and danger — that share identical padding, rounding, and font weight, but differ in color, and each needs a hover state that is visibly distinct from its resting state.

```
<button class="px-4 py-2">Primary</button>
<button class="px-4 py-2">Secondary</button>
<button class="px-4 py-2">Danger</button>
```

Your task:

1. Give all three buttons identical base spacing and rounding classes.
2. Style Primary with a solid blue background and white text.
3. Style Secondary as an outlined gray button with dark text.
4. Style Danger with a solid red background, and give all three a hover state one shade darker.

MEDIUM

Sticky Footer Layout

A page has a header, a main content area of variable length, and a footer. On pages with little content the footer should still sit at the bottom of the viewport; on pages with lots of content the footer should be pushed down naturally below it, never overlapping. Write plain CSS — no framework classes.

```
<body>
  <header>Site Header</header>
  <main>Page content of varying length.</main>
  <footer>Copyright 2026</footer>
</body>
```

Your task:

1. Write plain CSS rules (not Tailwind) so the footer sticks to the bottom of the viewport when content is short.
2. Make sure the footer is pushed down naturally, not overlapping, when content is long.
3. Do not use `position: fixed` for the footer.

MEDIUM

Holy Grail Layout with CSS Grid

A classic app shell needs a full-width header on top, a full-width footer on bottom, a 200px left navigation sidebar, a flexible main content area, and a 150px right sidebar — all built with plain CSS Grid template areas, no framework.

```
<div class="layout">
  <header class="header">Header</header>
  <nav class="nav">Nav</nav>
  <main class="main">Main</main>
  <aside class="aside">Aside</aside>
  <footer class="footer">Footer</footer>
</div>
```

Your task:

1. Define `.layout` as a CSS Grid with named template areas for header, nav, main, aside, and footer.
2. Make the header and footer span the full width above and below the three middle columns.
3. Size the nav column to 200px, the aside column to 150px, and let main take the remaining space.
4. Assign each child element to its named grid area.

MEDIUM

Flex Child Text Overflow Bug

A file-row component shows an icon and a filename. The filename should truncate with an ellipsis when too long, and `truncate`` is already applied, but the long filename still overflows the fixed-width row and breaks the layout instead of truncating.

```
<div class="flex items-center gap-2 w-64 border p-2">
  <svg class="w-5 h-5 shrink-0"></svg>
  <span class="truncate">this-is-a-very-long-filename-that-should-truncate.pdf</span>
</div>
```

Your task:

1. Explain why `truncate` is not taking effect on the flex child even though it is applied.
2. Add exactly one utility class to fix the truncation.
3. Keep the icon from shrinking when the filename is long.

MEDIUM

Responsive Bootstrap Navbar

A marketing site needs a navbar that shows nav links inline on desktop but collapses behind a toggle button on mobile, built with Bootstrap 5 navbar components.

```
<nav>
  <a href="#">Brand</a>
  <a href="#">Home</a>
  <a href="#">About</a>
  <a href="#">Contact</a>
</nav>
```

Your task:

1. Build the navbar using Bootstrap's navbar, navbar-brand, and collapse components.
2. Add a toggler button that shows only below the large breakpoint.
3. Put the three nav links inside a properly structured nav-item/nav-link list that collapses on mobile.

MEDIUM

A small green online-status dot should sit at the bottom-right corner of a circular avatar, slightly overlapping its edge, with a white ring separating it from the avatar image behind it. Currently the badge just renders inline below the avatar.

Your task:

1. Position the badge at the bottom-right corner of the avatar circle.
2. Add a white border ring around the badge so it stands out from the avatar behind it.
3. Use relative/absolute positioning utilities, not manual margins.

MEDIUM

Sticky Table Header on Scroll

A data table lives inside a scrollable panel with a bounded height. As users scroll through many rows, the column header row should stay pinned to the top of the panel instead of scrolling out of view.

```
<div class="max-h-96 overflow-y-auto">
  <table class="w-full">
    <thead>
      <tr><th class="p-2">Name</th><th class="p-2">Score</th></tr>
    </thead>
    <tbody>...</tbody>
  </table>
</div>
```

Your task:

1. Make the header row stick to the top of the scrollable panel while the body scrolls beneath it.
2. Give the header a background color so scrolled rows don't show through underneath it.
3. Keep the scrollable container's bounded height and overflow behavior.

CSS

MEDIUM

Responsive Typography for a Hero Heading

A marketing hero heading is a single fixed large size (`text-5xl`), which wraps awkwardly on mobile screens and looks too small relative to the hero image on large desktop monitors.

```
<h1 class="text-5xl font-bold">Build faster with our platform</h1>
```

Your task:

1. Apply a smaller base font size for mobile.
2. Scale the heading up at the sm and lg breakpoints in ascending order.
3. Constrain the heading's line length with a max-width utility so lines don't run too wide on large screens.

MEDIUM

Consistent Vertical Spacing in a Form

A stacked form has inconsistent gaps between its field groups because each field div carries a different, ad-hoc margin class (or none at all).

```
<form>
  <div class="mb-1"><label>Name</label><input class="border w-full"/></div>
  <div><label>Email</label><input class="border w-full"/></div>
  <div class="mb-5"><label>Password</label><input class="border w-full"/></div>
</form>
```

Your task:

1. Remove the inconsistent per-field margin classes.
2. Apply a single spacing utility to the form so every field group gets uniform vertical spacing automatically.
3. Explain which siblings the spacing utility actually applies margin to.

MEDIUM

Bootstrap Responsive Column Layout

A product listing page needs cards that show 1 per row on mobile, 2 per row on tablets, and 3 per row on desktop, using the Bootstrap grid system.

```
<div class="row">
  <div class="col"><div class="card">Product A</div></div>
  <div class="col"><div class="card">Product B</div></div>
  <div class="col"><div class="card">Product C</div></div>
</div>
```

Your task:

1. Wrap the row in a container.
2. Apply column classes so cards are full-width on mobile, 2-per-row on medium screens, and 3-per-row on large screens.
3. Add gutter spacing between the cards.

MEDIUM

Flex Items Not Wrapping to a New Line

A tag/chip list inside a fixed-width container should wrap onto multiple lines as more tags are added, but currently all tags are being squeezed onto a single line and overflow horizontally past the container border.

```
<div class="flex gap-2 w-80 border p-2">
  <span class="px-3 py-1 bg-gray-200 rounded-full">Design</span>
  <span class="px-3 py-1 bg-gray-200 rounded-full">Engineering</span>
  <span class="px-3 py-1 bg-gray-200 rounded-full">Marketing</span>
  <span class="px-3 py-1 bg-gray-200 rounded-full">Sales</span>
  <span class="px-3 py-1 bg-gray-200 rounded-full">Support</span>
</div>
```

Your task:

1. Identify the default flex behavior causing the overflow.
2. Fix the container so tags wrap onto additional lines instead of overflowing.
3. Ensure there is visible gap between wrapped lines, not just between items on the same line.

MEDIUM

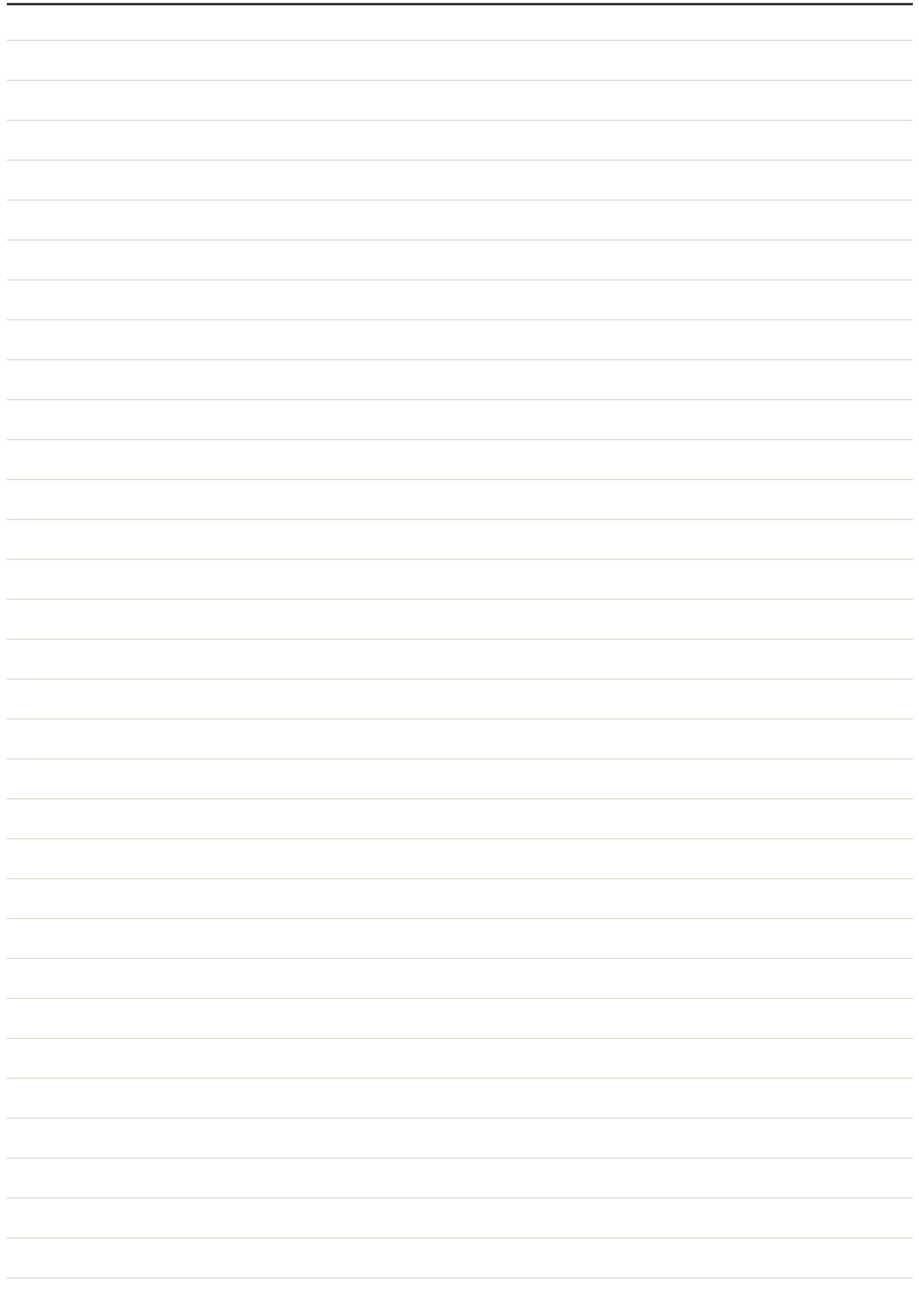
Dashboard Layout with Named Grid Areas

An analytics dashboard needs a fixed-width sidebar spanning the full height, a topbar across the remaining width, a large main chart area, and two smaller stacked stat widgets to the right of the chart — all with plain CSS Grid, no framework.

```
<div class="dashboard">
  <div class="sidebar">Sidebar</div>
  <div class="topbar">Topbar</div>
  <div class="main">Chart</div>
  <div class="stat1">Stat 1</div>
  <div class="stat2">Stat 2</div>
</div>
```

Your task:

1. Write a grid-template-areas layout where the sidebar spans the full height on the left.
2. Make the topbar span the remaining width above the chart and stat widgets.
3. Make the main chart area span two rows on the left of the remaining content, with stat1 and stat2 stacked in a narrower column on the right.
4. Assign each element its grid-area.



MEDIUM

Uniform Aspect-Ratio Image Cards

A product image grid looks jagged because source images have inconsistent aspect ratios. Every image should be forced into a consistent square shape and cropped to fill the box without stretching or distorting.

```
<div class="grid grid-cols-3 gap-4">
  
  
  
</div>
```

Your task:

1. Force each image into a consistent 1:1 square aspect ratio.
2. Ensure the image fills the square box completely by cropping rather than stretching.
3. Keep the grid responsive by not using a fixed pixel height.

MEDIUM

Modal Overlay Centering with Z-Index

A confirmation modal should appear centered on screen above a dimmed backdrop covering the entire page. Currently the modal renders at the top-left of its container and the backdrop does not cover the full page. Write plain CSS.

```
<div class="overlay">
  <div class="modal">Are you sure you want to delete this item?</div>
</div>
```

Your task:

1. Write CSS for `.overlay` so it covers the full viewport with a dimmed, semi-transparent background regardless of page scroll position.
2. Center `.modal` within the overlay using Flexbox.
3. Make sure the overlay stacks above all other page content using z-index.

MEDIUM

A settings list of options should show a thin horizontal divider line between each row, but no divider above the first item or below the last one. Manually adding borders to every list item would put an extra line at the bottom.

Your task:

1. Add dividers between the list items only, not around the outside of the list, using a single utility on the parent.
2. Keep the existing outer border and rounded corners on the list.
3. Specify a visible divider color.

MEDIUM

Equal-Height Pricing Card Grid

Three pricing cards in a Bootstrap row have different amounts of text, so they currently render with mismatched heights, looking uneven. They should all stretch to match the tallest card in the row.

```
<div class="row">
  <div class="col-md-4"><div class="card"><div class="card-body">Short text</div></div></div>
  <div class="col-md-4"><div class="card"><div class="card-body">A much longer paragraph of text that wraps ac
  <div class="col-md-4"><div class="card"><div class="card-body">Medium length text</div></div></div>
</div>
```

Your task:

1. Make all three cards stretch to equal height without hardcoding a pixel height.
2. Rely on Bootstrap's row/column flex behavior rather than a fixed-height hack.
3. Keep consistent gutter spacing between the cards.

MEDIUM

Equal Height Columns Without Explicit Heights

Three sidebar/content/aside columns with differing content lengths need matching background heights so the layout looks like solid equal-height columns, without hardcoding any pixel heights. Write plain CSS.

```
<div class="columns">
  <div class="col-a">Short</div>
  <div class="col-b">Much longer content spanning several lines of text.</div>
  <div class="col-c">Medium length content</div>
</div>
```

Your task:

1. Write plain CSS so all three columns automatically match the height of the tallest column.
2. Distribute the columns to share the row's width equally.
3. Explain why this works without any explicit height property.

CSS

MEDIUM

Container Not Centering on Wide Screens

A page section should be centered with a constrained max width on large monitors, but it stays flush against the left edge of the browser window no matter how wide the screen is.

```
<div class="max-w-4xl px-4">
  <h1>Welcome</h1>
  <p>Section content goes here.</p>
</div>
```

Your task:

1. Fix the class list so the section is horizontally centered in the viewport.
2. Keep the existing max-width constraint and inner padding.
3. Explain why the fix works and why margin-based centering needs a bounded width.

MEDIUM

Auto-Responsive Card Grid Without Media Queries

A card grid should automatically add or remove columns as the browser window is resized, without writing any breakpoint-specific media queries, while keeping every card at least 220px wide. Write plain CSS.

```
<div class="card-grid">
  <div class="card">Card 1</div>
  <div class="card">Card 2</div>
  <div class="card">Card 3</div>
  <div class="card">Card 4</div>
</div>
```

Your task:

1. Write a single grid-template-columns declaration that produces fluid, responsive columns with no media queries.
2. Ensure each card is never narrower than 220px.
3. Add consistent spacing between cards.

MEDIUM

Aligned Label-Input Form Rows

A settings form has label/input pairs arranged in flex rows, but labels of different text lengths (like 'Email Address' versus 'Phone') push the inputs out of alignment, so the input column looks ragged instead of forming a clean vertical line.

```
<form>
  <div class="flex gap-2"><label>Name</label><input class="border flex-1"/></div>
  <div class="flex gap-2"><label>Email Address</label><input class="border flex-1"/></div>
  <div class="flex gap-2"><label>Phone</label><input class="border flex-1"/></div>
</form>
```

Your task:

1. Give every label a consistent fixed width so the inputs align into a clean column.
2. Prevent longer label text from shrinking below the fixed width.
3. Keep the inputs filling the remaining row width.

MEDIUM

Multi-Line Text Truncation for Card Descriptions

Blog preview cards have descriptions of varying length. Each card's description should show at most 3 lines with an ellipsis when the text is longer, so cards in the same row stay a uniform height.

```
<div class="grid grid-cols-3 gap-4">
  <div class="p-4 border rounded">
    <h3 class="font-bold">Title</h3>
    <p>A long description that could run to many lines and break the card grid's uniform height across the row</p>
  </div>
</div>
```

Your task:

1. Constrain the description paragraph to a maximum of 3 lines with an ellipsis when truncated.
2. Explain why a single-line truncate utility would not satisfy this requirement.
3. Keep the surrounding grid layout unchanged.

MEDIUM

Bootstrap Toolbar Alignment

A document library toolbar should show a title on the far left and two action buttons grouped together on the far right, vertically centered. Currently the title and buttons are stacked vertically on the left because no flex utilities are applied.

```
<div class="toolbar p-2 border-bottom">
  <h5>Documents</h5>
  <button class="btn btn-primary">New</button>
  <button class="btn btn-outline-secondary">Import</button>
</div>
```

Your task:

1. Use Bootstrap flex utilities to push the title to the left and the button group to the right.
2. Keep the two buttons grouped together as a single unit rather than spread apart.
3. Vertically center all items in the toolbar, and add a small gap between the two buttons.

MEDIUM

Button with Dark Mode and Focus States

A primary 'Save' button needs to look correct in both light and dark mode, with a visibly distinct hover state and a keyboard-accessible focus ring that only appears for keyboard navigation, not for mouse clicks.

```
<button class="px-4 py-2 rounded bg-blue-600 text-white">Save</button>
```

Your task:

1. Add a hover state that is a visibly darker shade than the base color.
2. Add a dark-mode variant that adjusts the background color appropriately.
3. Add a focus ring that only appears for keyboard focus, not for mouse clicks.

CSS

MEDIUM

Sticky Sidebar That Scrolls With Content

A documentation page has a table-of-contents sidebar next to a long article. The sidebar should stay pinned near the top of the viewport while the article scrolls, but should not overlap the footer once the article ends.

```
<div class="flex gap-8">
  <aside class="w-64">Table of contents links here.</aside>
  <article class="flex-1">Long article content spanning many screens of scrolling.</article>
</div>
```

Your task:

1. Make the sidebar stick near the top of the viewport while scrolling through the article, offset 1rem from the top.
2. Fix whatever default flex behavior would otherwise prevent the sticky positioning from having any visible effect.
3. Leave the article column's layout unaffected.

CSS

MEDIUM

Featured Item Spanning Multiple Grid Cells

A news homepage grid has 3 columns. The lead story should stand out by spanning 2 columns and 2 rows, while the remaining smaller stories fill single cells around it.

```
<div class="grid grid-cols-3 gap-4">
  <div class="border p-4">Featured Story</div>
  <div class="border p-4">Story 2</div>
  <div class="border p-4">Story 3</div>
  <div class="border p-4">Story 4</div>
  <div class="border p-4">Story 5</div>
</div>
```

Your task:

1. Make the first item span 2 columns and 2 rows.
2. Keep the remaining story items as normal single grid cells.
3. Keep the 3-column base grid layout.

CSS

MEDIUM

Navbar Items Misaligned Despite justify-between

A navbar has a logo and a group of nav links spread to opposite ends with `justify-between`, but the logo image sits visibly higher than the baseline of the link text next to it.

```
<nav class="flex justify-between p-4 border-b">
  
  <div class="flex gap-4">
    <a href="#">Home</a>
    <a href="#">Pricing</a>
    <a href="#">Contact</a>
  </div>
</nav>
```

Your task:

1. Fix the vertical misalignment between the logo and the nav links.
2. Do not change the horizontal spacing behavior already provided by justify-between.
3. Explain which flex axis controls this alignment versus which axis justify-between controls.

CSS

MEDIUM

Responsive Padding for a Hero Section

A hero section looks cramped on mobile because of minimal horizontal padding, and disproportionately padded on large desktop screens because of one large fixed vertical padding value applied at every screen size.

```
<section class="px-2 py-24 bg-indigo-600 text-white">
  <h1>Welcome to our platform</h1>
  <p>Get started in minutes.</p>
</section>
```

Your task:

1. Increase horizontal padding on mobile for better breathing room.
2. Scale both horizontal and vertical padding up across the sm and lg breakpoints in ascending order.
3. Ensure the base (mobile) vertical padding is smaller than the desktop value, rather than one fixed large value everywhere.

